

Progress Report

MFJA 3rd Floor Gazebo / ROS 2 Simulation

Reporting period: Week 10

Overview

During Week 10, the main work focused on improving the ROS 2 communication layer for the Room 315 rail simulation and adding a clean way to run one industrial robot independently. The progress centered on making the simulation more modular, easier to test, and easier to extend. A major part of the work was the transition from loosely defined topic data to dedicated typed message interfaces in the `mfja_rail_interfaces` package.

The updated structure gives each rail subsystem a clear command and state interface. Switches, stoppers, shuttles, and sensors now use explicit ROS 2 message types, which improves readability, reduces ambiguity during testing, and prepares the system for later integration with controllers and planners.

Typed Message Interface Improvements

The `mfja_rail_interfaces` package was updated with the typed messages required for Room 315 rail control and feedback. The important message files include:

- `NamedState.msg`
- `SwitchCommand.msg` and `SwitchState.msg`
- `StopperCommand.msg` and `StopperState.msg`
- `ShuttleCommand.msg` and `ShuttleState.msg`
- `SensorReading.msg` and `SensorFeedback.msg`

The switch command and state messages were simplified around a shared named-state pattern. Each switch message carries a ROS header and a list of named switch states:

```
std_msgs/Header header
NamedState[] switches
```

This makes it possible to command a single switch, such as `A1`, or to send a group command using `ALL`. The same idea was used for stopper commands and states, giving the system a consistent model for routing and blocking devices.

The shuttle interface was also improved. `ShuttleCommand` now supports the shuttle name, command, start slot, and speed. This allows commands such as turning a shuttle on, turning it off, resetting it, removing it, or adding a new shuttle during runtime. `ShuttleState` reports the mode, current segment, path coordinate, Cartesian position, yaw angle, and speed.

Sensor feedback was structured through `SensorReading` and `SensorFeedback`. Each reading can include the sensor name, type, active state, related shuttle, rail segment, normalized position, and distance. This is useful for validating approach and position sensors on both rail networks.

Topic Organization and Testing

The Room 315 topics were organized under clear rail namespaces. The right and left rail systems now follow the same topic structure:

```
/room_315/rails/right/...  
/room_315/rails/left/...
```

Examples of the updated typed topics are:

- /room_315/rails/right/switches/command
- /room_315/rails/right/switches/state
- /room_315/rails/right/stoppers/command
- /room_315/rails/right/stoppers/state
- /room_315/rails/right/shuttles/command
- /room_315/rails/right/shuttles/state
- /room_315/rails/right/sensors/feedback
- /room_315/rails/right/sensors/position_feedback

This structure makes it easier to test each rail side separately and debug the system with standard ROS 2 commands such as `ros2 topic list`, `ros2 topic echo`, and `ros2 topic pub`. The documentation was updated with practical commands for launching Room 315, publishing switch and stopper commands, controlling shuttles, adding shuttles during runtime, and checking sensor feedback.

Single Industrial Robot Launch Mode

Another important achievement was the addition of an isolated launch mode for running one industrial robot by itself. This mode loads only the ground plane, one selected industrial robot, and the matching support table. It does not load Room 315, rails, shuttles, sensors, fixtures, other robots, or TIAGo.

The main launch entry point is:

```
ros2 launch mfja_3rd_floor_bringup single_industrial_robot.launch.py \  
  robot:=kuka gui:=true start_paused:=false
```

The supported robot selectors include:

- kuka
- Staubli
- hc10
- hc10dt

The launch system selects the requested robot from the configuration, resolves its SDF and URDF assets, spawns the proper support table, starts `robot_state_publisher`, and creates the required ROS–Gazebo bridge topics for joint trajectories, joint states, and trajectory progress. It also uses a Gazebo partition to isolate the launch instance from other simulations.

Conclusion

Overall, the work completed during this period made the simulation cleaner and more reliable. The rail system now has typed ROS 2 interfaces, better topic organization, clearer command/state separation, and structured sensor feedback. The isolated robot launch mode also allows individual industrial robots to be tested without loading the full floor environment. These improvements reduce debugging time and create a stronger base for future work on robot control, rail automation, and full-system integration.